

Relatório Técnico

Diagnóstico de Telemetria em Assembly ARM64

Sistema CLI stand-alone implementado em Assembly ARM64 para processar dados internos de telemetria, gerar métricas escalares, manipular campos de bits, aplicar LUT, executar rotinas SIMD NEON inteiras e de ponto flutuante, e emitir um relatório compacto em stdout. A validação foi feita no QEMU, com evidências de execução, GDB, objdump e readelf.

ALUNO Renato DISCIPLINA DR4 - Processamento Otimizado em Assembly ARM 64-bits ENTREGA Assessment Final	AMBIENTE DE VALIDAÇÃO aarch64-linux-gnu-as/ld + qemu-aarch64 + gdb-multiarch ALVO PREVISTO Raspberry Pi Zero 2W / Linux ARM64 ARTEFATOS Código Assembly, Makefile, evidências textuais e relatório
--	--

1. Objetivo e escopo

O objetivo do AT foi transformar os 12 exercícios integrados em um único sistema funcional, reproduzível e auditável. O executável final usa apenas Assembly ARM64, sem libc, recebe os dados a partir da memória interna do programa e escreve o relatório operacional por syscalls Linux diretas.

Os dados de entrada seguem o enunciado: valores 10 20 30 40 50 60 70 80 e status 0x01 0x01 0x05 0x01 0x09 0x01 0x00 0x01. Apenas amostras com bit 0 ativo entram nas métricas estatísticas.

Decisão de ambiente: embora o alvo seja o Raspberry Pi Zero 2W, o professor liberou a validação em QEMU. O projeto, portanto, gera um ELF AArch64 stand-alone e estático, com símbolos de depuração, executado e depurado por qemu-aarch64 e gdb-multiarch.

2. Organização do projeto

A estrutura foi separada em módulos com responsabilidade explícita, exatamente para atender ao enunciado e aos critérios de organização, linking e auditoria.

- Assembly ARM64
- Stand-alone
- Syscalls Linux
- LUT
- NEON SIMD
- Macros GAS
- QEMU + GDB

Arquivo / pasta	Função
src/main.s	Ponto de entrada _start, integração do fluxo e emissão do relatório por stdout.
src/processamento.s	Métricas escalares, contagem de flags, recomposição/rotação de campos, LUT e rotinas NEON.
src/conversao.s	Conversão inteiro-string, hexadecimal fixo, strlen e parser validado.
src/macros.inc	Macros GAS parametrizadas para carregar endereços, escrever em stdout e encerrar o programa.
Makefile	Automatiza montagem, linking, execução, teste por diff, evidências, GDB e relatório.
tests/expected_output.txt	Saída esperada usada por make test para validação byte a byte.

Fluxo de validação

1. Build

aarch64-linux-gnu-as monta cada módulo e aarch64-linux-gnu-ld gera o ELF final.

2. Execução

qemu-aarch64 executa o binário AArch64 sem depender de libc.

3. Teste

A saída real é comparada com tests/expected_output.txt.

4. Auditoria

objdump, readelf e GDB registram instruções, símbolos e registradores.

```
make
make test
make evidence
make gdb-evidence
make report
```

3. Implementação dos exercícios

Ex.	Implementação	Evidência técnica
1	Projeto dividido em main.s, processamento.s, conversao.s e macros.inc.	Separação física dos módulos no diretório src/.
2	Makefile com make e make clean, além de alvos de teste e evidência.	Build reproduzível por um comando.
3	GDB remoto via qemu-aarch64 -g, com breakpoints em métricas, LUT e NEON.	evidencias/gdb_registers.txt.
4	Análise por objdump/readelf com seções, símbolos e instruções escalares/vetoriais.	objdump_full.txt e readelf_sections_symbols.txt.
5	Loop escalar conta válidos, soma valores válidos e calcula média inteira com udiv.	Resultados: 7, 290, 41.
6	u64_to_ascii, strlen_ascii, u64_to_hex_fixed e parse_u64_checked.	Conversões usadas no relatório e validação interna de 290.
7	Bits de alarme/bateria por tst, campos por and/orr/lsl, rotação por lsl/lsl/orr, assinatura por eor e limpeza por bic.	0x1117, 0x22e2, 0x33f5.
8	LUT em memória com indexação valor / 10 - 1 e acesso ldr [base, index, uxtw #2].	Saída: 100 200 300 400 500 600 800.
9	NEON inteiro carrega 10 20 30 40 em q0 e reduz com addv s1, v0.4s.	Resultado: 100.
10	NEON float32 carrega 100.0 200.0 300.0 400.0 e divide por 1000.0 com fdiv v2.4s.	Memória: 0.100000001 0.200000003 0.300000012 0.400000006.
11	Macros GAS parametrizadas WRITE_STDOUT, WRITE_REG, EXIT_PROGRAM e LOAD_ADDR.	Uso direto em main.s para syscalls de saída e encerramento.
12	Todas as rotinas são chamadas por _start e integradas em um único relatório operacional.	Execução final validada por make test.

4. Cobertura da rubrica

Critério da rubrica	Como foi demonstrado
Infraestrutura, múltiplos arquivos, Makefile, GDB e objdump	Projeto em módulos Assembly, Makefile reproduzível, GDB remoto e desassemblagem/readelf preservados em evidências.
Aritmética e conversão numérica	Métricas com soma/média inteiras, parser validado, conversão decimal e hexadecimal para stdout.
Bits, campos, rotações e LUT	Contadores por bits de status, recomposição de palavra compacta, rotação manual, assinatura por EOR e LUT indexada.
SIMD NEON inteiro e ponto flutuante	addv v0.4s para soma inteira e fdiv v2.4s para normalização float32.
Macros GAS e funções de string	Macros parametrizadas para syscalls e funções utilitárias seguras em conversao.s.
Stand-alone sem libc e integração final	Link com ld -static, ponto de entrada próprio _start e syscalls Linux diretas.

5. Validação e evidências

A validação funcional principal compara a saída completa do executável com uma saída esperada versionada em `tests/expected_output.txt`. Se qualquer byte divergir, `make test` falha.

Saída final do programa

```
DR4 AT - Diagnostico de Telemetria
entrada valores: 10 20 30 40 50 60 70 80
entrada_status: 0x01 0x01 0x05 0x01 0x09 0x01 0x00 0x01
amostras_validas: 7
soma_validos: 290
media_inteira: 41
alarmes_ativos: 1
bateria_baixa: 1
palavra_status: 0x1117
rotacao_status: 0x22e2
assinatura_status: 0x33f5
lut_validos: 100 200 300 400 500 600 800
soma_lut_validos: 2900
simd_soma4: 100
simd_normalizado: 0.100 0.200 0.300 0.400
validacao_numerica: ok
```

Binário gerado

```
build/dr4_at: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV),
statically linked, with debug_info, not stripped
```

GDB remoto

```
[metrics_done]
x0 = 7          x1 = 290      x2 = 41
x3 = 1          x4 = 1        x5 = 0x1117
x6 = 0x33f5

[lut_done]
x0 = 7          x1 = 2900
lut_output_values = 100 200 300 400 500 600 800

[neon_int_done]
x0 = 100
v0.s = {0x0a, 0x14, 0x1e, 0x28}

[neon_float_done]
normalized_values = 0.100000001 0.200000003 0.300000012 0.400000006
```

Objdump

```
and x12, x3, #0xf
orr x12, x12, x13
lsl x13, x13, #4
lsr x14, x12, #11
bic x16, x15, x12
eor x14, x12, x13
addv s1, v0.4s
fdiv v2.4s, v0.4s, v1.4s
svc #0x0
```

Os arquivos completos estão em `evidencias/qemu_output.txt`, `evidencias/gdb_registers.txt`, `evidencias/objdump_full.txt`, `evidencias/objdump_processamento.txt` e `evidencias/readelf_sections_symbols.txt`.

6. Conclusão

A entrega atende ao enunciado do AT e aos itens de rubrica: o projeto é modular, stand-alone, reproduzível por Makefile, auditável por `objdump/readelf`, depurável por GDB e validado por execução AArch64 em QEMU. O sistema integra processamento escalar, conversão textual, manipulação de bits, LUT, SIMD inteiro, SIMD float32, macros GAS e syscalls Linux diretas.

A validação final passou sem divergências no teste por diff e as evidências preservadas permitem reproduzir os resultados e inspecionar o comportamento do binário em pontos internos relevantes.